

基于概率需求建模的 LLM 应用高效服务

Yifei Liu

Shanghai Jiao Tong University
China

Zuo Gan

Shanghai Jiao Tong University
China

Zhenghao Gan

Shanghai Jiao Tong University
China

Weiye Wang

Shanghai Jiao Tong University
China

Chen Chen*

Shanghai Jiao Tong University
China

Yizhou Shan

Huawei Cloud
China

Xusheng Chen

Huawei Cloud
China

Zhenhua Han

Unaffiliated
China

Yifei Zhu

Shanghai Jiao Tong University
China

Shixuan Sun

Shanghai Jiao Tong University
China

Minyi Guo

Shanghai Jiao Tong University
China

摘要

基于大语言模型 (LLM) 的应用通过一系列任务协同解决现实问题, 从而增强模型的能力; 这些应用在不同后端上的资源需求量具有动态性。现有服务系统将 LLM 应用的资源需求视为黑箱, 因此不合理的排队顺序和后端预热延迟会损害端到端效率。我们发现, 可以利用概率需求图 (Probabilistic Demand Graph, PDGraph) 以通用且准确的方式建模 LLM 应用的资源需求。在此基础上, 我们提出 Hermes, 利用 PDGraph 高效服务 LLM 应用。面对概率化的需求描述, Hermes 使用 Gittins 策略确定调度顺序, 以尽量缩短应用的平均完成时间。Hermes 还利用 PDGraph 模型, 在恰当时刻提前预热处于冷状态的后端。针对多种 LLM 应用的实验表明, Hermes 能够有效提升应用服务效率, 将平均完成时间降低超过 70%, 将 P95 完成时间降低超过 80%。

1 引言

大语言模型 (LLM) [23, 30, 74] 已经展现出强大的语言理解和生成能力。然而, 尽管 LLM 正在快速发展 [52, 62, 72], 人们普遍认为, 单次 LLM 请求往往不足以解决许多现实问题 [11]。例如, 典型 LLM 的输入上下文窗口大小通常有限 (例如, 少于 10M 个 Token) [26, 50], 因此处理大型文档时需要发起多个并行推理请求。同时, LLM 请求的输出可能不可靠, 即出现幻觉 [42]; 因此, 需要额外的 LLM 请求来保证输出质量, 例如进行自我反思 [43]。此外, LLM 的内置知识和交互方式也存在限制, 因此系统往往会集成 Docker 执行 [4, 75] 或第三方工具调用 [68, 69] 等非 LLM 任务, 以增强 LLM 的能力 [17]。我们将这样一组相互关联、共同解决现实问题的 LLM 任务和非 LLM 任务称为一个 LLM 应用。LLM 应用将成为未来 AI 工作负载的主流范式。

LLM 应用通常托管在云端 [2, 40, 51], 因此高效服务这些应用至关重要: 应当使应用尽快完成, 让用户及时获得有效的最终输出。不过, 与操作系统和大数据领域的

传统工作负载相比, LLM 应用具有两个不同的特点。首先, LLM 应用的资源需求, 例如每次请求生成的 Token 长度以及请求之间的结构, 取决于运行时输入, 事先并不确定。由于难以获知应用的总需求量, vLLM [13] 和 Parrot [51] 等现有服务系统选择 FCFS 等简单调度算法; 队头阻塞问题会因此降低调度效率。其次, 服务 LLM 应用往往涉及多种后端资源, 例如用于代码测试的 Docker 容器 [4], 以及用于加速推理的 KV cache [74]; 其中许多资源都是按需准备的 [33]。因此, 冷后端间歇性发生的预热延迟可能拖长应用完成时间。综上, 应用需求信息的缺失, 使现有 LLM 服务系统在队列管理和后端准备两个方面都缺乏效率。

高效服务 LLM 应用的关键, 在于获取准确的需求信息。事实上, 尽管 LLM 应用的需求具有显著动态性, 这并不意味着服务系统必须忽略需求: 从应用角度观察, 能够发现有前景的需求感知机会。典型 LLM 应用由多个功能单元组成, 例如使用固定系统提示词来验证刚生成声明的推理任务。由于具有特定功能特征, 同一个功能单元在不同运行中的资源需求相对稳定。LLM 应用通常会重复执行, 且代码文件托管在云端, 因此可以通过静态和动态程序分析来建模其资源需求。

然而, 要为一般性的 LLM 应用准确建模需求, 并非易事。不同应用可能涉及不同类型的后端, 也可能具有不同的功能单元结构; 我们需要设计合适的需求描述原语, 以支持通用性。与此同时, 在每次应用运行中, 用户输入会影响哪些功能单元被触发; 每个功能单元的需求量也可能偏离此前分析得到的平均值。因此, 任何固定的需求表示都过于武断; 我们还需要在线细化估计, 从而更准确地估计当前运行中的资源需求。

本文设计了 Hermes, 一个用于高效服务 LLM 应用的系统。在 Hermes 中, 我们提出将 LLM 应用的资源需求建模为概率需求图 (PDGraph)。PDGraph 用图结构组织 LLM 应用中各种功能单元: 每个 PDGraph 节点使用分布函数描述相应功能单元的需求量, 同时记录下游依赖关系及各自对应的分支选择概率。此外, 通过分析上游单元和下游单元之间的需求相关性, 我们能够结合当前运

*Chen Chen 为通讯作者。

行的最新执行状态，不断细化需求估计。由此，PDGraph 能够以概率方式，忠实估计整个应用以及即将执行的单元的资源需求。随后，我们可以利用这些信息优化排队顺序，并决定何时准备后端。

在队列优化方面，我们面临的问题是：当应用需求以分布而非确定值表示时，应该如何确定调度顺序。此时，基于需求分布均值的经典最短剩余时间优先（SRTF）算法，已不再是最小化应用平均完成时间的最优方法。为了适应需求不确定性，Hermes 采用 Gittins 策略。Gittins 策略 [65] 在调度需求未知但需求分布已知的作业时，已被证明是最优的，因此非常适合调度 LLM 应用。该策略计算 Gittins 指数，用于在运行时估计应用实际的剩余处理时间。此外，我们还考虑每个应用都具有关联截止时间的情形：利用估计得到的需求分布信息，Hermes 采用最小松弛时间优先（LSTF）算法，优先处理违反截止时间风险更高的应用。

除了队列优化，Hermes 还利用 PDGraph 提前预热后端。在一个功能单元执行期间，利用 PDGraph 中记录的需求信息，我们可以在其下游单元到达之前，提前预热下游单元所需的后端。由于 LLM 应用中的功能单元可能具有多个下游单元，而它们的到达时刻也不确定，取决于当前单元的执行时间，因此，选择预热哪种后端以及何时预热，都存在明确的权衡。预热错误的后端，或者过早预热后端，都会浪费资源。为此，Hermes 引入一个可调参数，用于控制延迟降低效果与资源浪费之间的权衡。这种预热原则既可通用于 KV cache 等 LLM 后端，也可通用于 Docker 容器等非 LLM 后端。

我们使用超过 4,000 行 Python 代码实现了 Hermes，并构建了包含代表性 LLM 应用的工作负载套件。我们使用多种 LLM 应用，比较 Hermes 与主流服务系统的性能。实验结果表明，Hermes 能够将应用平均完成时间降低超过 70%；在具有显式截止时间的情形下，还能取得超过 1× 的改进。此外，消融研究进一步证实了 Hermes 缓解需求不确定性、并为多种后端高效供给资源的有效性。

本文的贡献如下：

- 我们识别出现有系统在服务 LLM 应用时的局限性。这类应用具有需求动态、后端多样的特点，而现有系统会产生较长的排队延迟和后端预热延迟。
- 我们提出利用 PDGraph，以概率化、结构化的方式建模 LLM 应用需求，并进一步设计 Hermes，利用 PDGraph 优化排队顺序和后端准备。
- 我们构建了包含典型 LLM 应用的工作负载套件，并通过测试平台实验验证 Hermes 的有效性，证明它相对于现有系统具有显著的效率提升。

2 背景与动机

2.1 LLM 应用概述

LLM 应用。大语言模型（LLM）[23, 30, 74] 拥有内置的世界知识，有望应用于金融 [49]、艺术 [53] 和科学 [63] 等多种领域。由于训练成本高昂，且可用数据语料来源逐渐耗尽，通过横向扩展而非纵向扩展增强现有 LLM 能力的趋势正在兴起 [11]。也就是说，为了扩展输入长度 [44, 50]、提高输出可靠性 [27, 43]，或者集成更多功

能 [68, 69]，LLM 实践者往往需要联合执行多个 LLM 推理任务以及必要的非 LLM 任务，从而形成复合任务工作流；我们将其称为 LLM 应用。为了具体说明，我们构建了一个覆盖广泛的 LLM 应用工作负载套件，如图 1 所示。该套件包括十种代表性应用，它们使用特定框架，例如 FacTool [27]，或者遵循特定控制策略，例如 Graph of Thought [22] 和 ReAct [43]。

特别地，我们注意到，LLM 应用与操作系统或大数据领域传统工作负载的差异，体现为需求动态性和后端多样性。首先，LLM 应用的执行状态高度动态：由于 ReActing [43, 78] 和 LLM 规划 [68] 等机制，任务之间的结构只能在运行时确定；同时，每个 LLM 任务的输入与输出 Token 长度也都是事先未知的 [33, 77]。其次，服务 LLM 应用需要供给多种 LLM 后端和非 LLM 后端。对于 LLM 后端，不同应用可能偏好不同的基础模型或微调适配器 [25, 38]。对于非 LLM 后端，同样可能需要多种后端：例如，在代码生成应用中，LLM 生成的代码需要在 CPU 后端上测试，可使用 Docker 容器 [4, 75] 提供隔离环境；同时，HuggingGPT 应用 [10] 可能需要加载多种非 LLM 模型，将它们作为工具使用。

LLM 应用的云端服务。与 OpenAI assistant API [2] 类似，LLM 应用通常托管在云端，以便用户访问 [1, 40, 51, 71]。这些应用的代码文件由服务提供方维护，在用户提交应用输入后启动。当服务来自不同用户的多个 LLM 应用时 [40, 71]，服务提供方需要先将到达任务组织在全局队列中，然后分别将每个任务分发给合适的 LLM 后端或非 LLM 后端。良好的服务系统应当同时实现较高的执行效率和资源利用率。具体而言，启动 LLM 应用时，用户主要关注应用完成时间，也就是高质量输出，例如有效代码，最终返回的时刻 [51]。因此，应当在应用层面设置效率目标，即最小化应用平均完成时间（ACT）；若提供了期望截止时间，则最大化有效吞吐量，也就是在截止时间之前完成的应用数量。同时，为了提高资源利用率，LLM 服务提供方应当避免在服务 LLM 应用时闲置地占用资源。

2.2 现有实践及其局限性

对于 LLM 应用，由于需求动态性强且后端种类多样，现有服务系统往往无法取得较高的调度效率。

首先，面对不确定的资源需求，现有实践通常在任务排队时采用 FCFS 等简单启发式方法，而这些方法的低效性已经为人所知。当下主流 LLM 服务框架 vLLM [13] 使用 FCFS 调度 LLM 推理任务，并不知道更高层面 LLM 应用的存在；在资源竞争时，同一个 LLM 应用的组成任务可能被其他应用的任务穿插，从而导致应用完成延迟。近期，Parrot [51] 提出将一个应用的推理任务一起调度；然而，由于缺乏具体的资源需求量信息，它仍然按照 FCFS 调度 LLM 应用，因此受到队头阻塞问题影响。我们的测试平台测量表明，先执行大型应用，即图 1 (a) 中的 DM，再执行小型应用，即图 1 (i) 中的 KBQAV，平均 ACT 为 52.8 秒；若颠倒执行顺序，则仅为 40.3 秒。另一项工作 VTC [71] 提出在不同应用，即不同租户之间公平分配计算能力，以避免队头阻塞；但在资源竞争时，

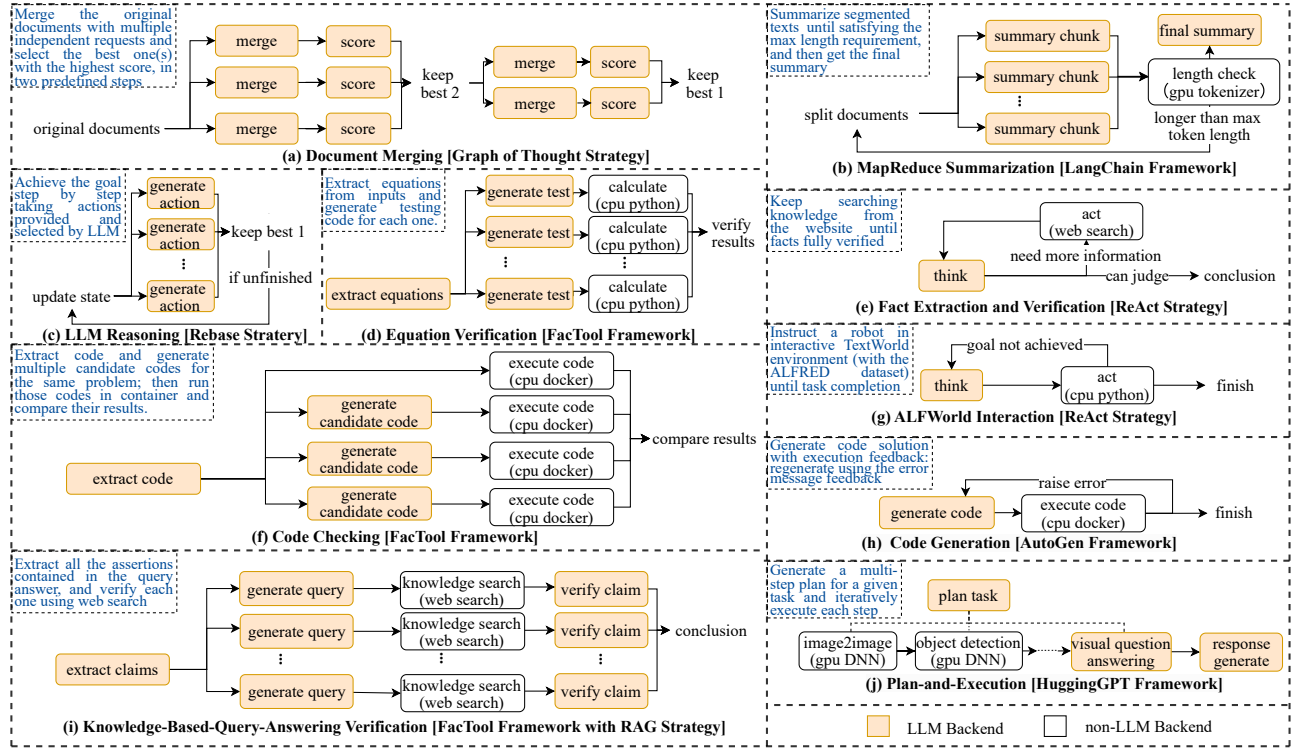


图 1. 十种具有代表性的 LLM 应用：(a) 文档合并 (DM) [6]；(b) MapReduce 摘要 (MRS) [12]；(c) LLM 推理 (LLMR) [32]；(d) 方程验证 (EV) [7]；(e) 事实提取与验证 (FEV) [8]；(f) 代码检查 (CC) [4]；(g) ALFWorld 交互 (ALFWI) [3]；(h) 代码生成 (CG) [5]；(i) 基于知识的问答验证 (KBQAV) [9]；(j) 规划与执行 (PE) [10]。

它强制每个应用只能使用其公平份额，因此仍然效率较低。

其次，由于 LLM 应用可能在不确定的时刻调用多种后端，让这些后端始终处于待命状态并不高效。例如，HuggingGPT [68] 提供了 130 个官方输入示例；我们的测量显示，只有 20 次运行触发了文本到视频模型。为了避免资源浪费，这些服务后端应按需准备；但这会给不同后端带来冷启动延迟。例如，对于 LLM 后端，用户可能根据成本与质量偏好选择不同的基础模型 [25]，或者需要定制 LoRA 适配器 [38] 来获得特定领域的能力；同时，还可能通过复用 KV cache 来加速推理 [33, 47]。这些模型或缓存内容都需要在推理开始前预先加载至 GPU HBM。对于非 LLM 后端，代码生成应用需要在代码测试之前启动 Docker 容器；HuggingGPT 应用 [68] 则需要在任务执行之前，将目标识别等 DNN 模型加载至 GPU。这些准备过程可能耗费不可忽略的时间，拖慢应用的端到端完成。图 2a 和图 2b 展示了一系列后端内容的典型预热时间，最长可达到典型推理任务执行时间的 18x；该典型任务具有 1000 个输入 Token 和 100 个输出 Token。注意到 KV cache 预热的必要性，近期 CachedAttention 工作 [33] 提出，根据调度器的等待队列驱逐和预取 KV cache；我们将此方法称为基于等待队列的驱逐/预取 (EPWQ)。然而，EPWQ 无法为尚未积压在队列中的请求，例如尚未提交的未来请求，预取 KV cache；由于 EPWQ 缺乏应用

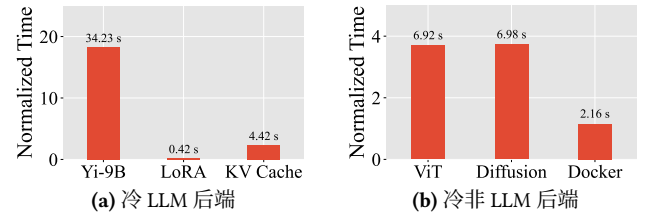


图 2. 冷后端的预热成本，归一化到具有 1000 / 100 个输入/输出 Token 的典型 LLM 推理任务执行时间。LoRA 用于 LLaMA-7B，秩为 8；KV cache 大小为 128K，加载至 A100 GPU。Docker 镜像为 python:3.10-slim。

视角，LLM 应用的下游任务即使具有最高优先级，也可能在到达之后承受 KV cache 加载延迟。

总而言之，面对不同后端上的动态需求，现有 LLM 服务系统不得不采用静态排队策略，并以被动响应方式供给资源。这种不感知需求的服务方式损害了 LLM 应用的整体效率。

2.3 洞察与挑战

洞察。 提高 LLM 应用服务效率的关键，在于提前获取其需求信息。虽然 LLM 应用的需求具有动态性，但我们注意到，这并不意味着它们完全不可预测。事实上，LLM

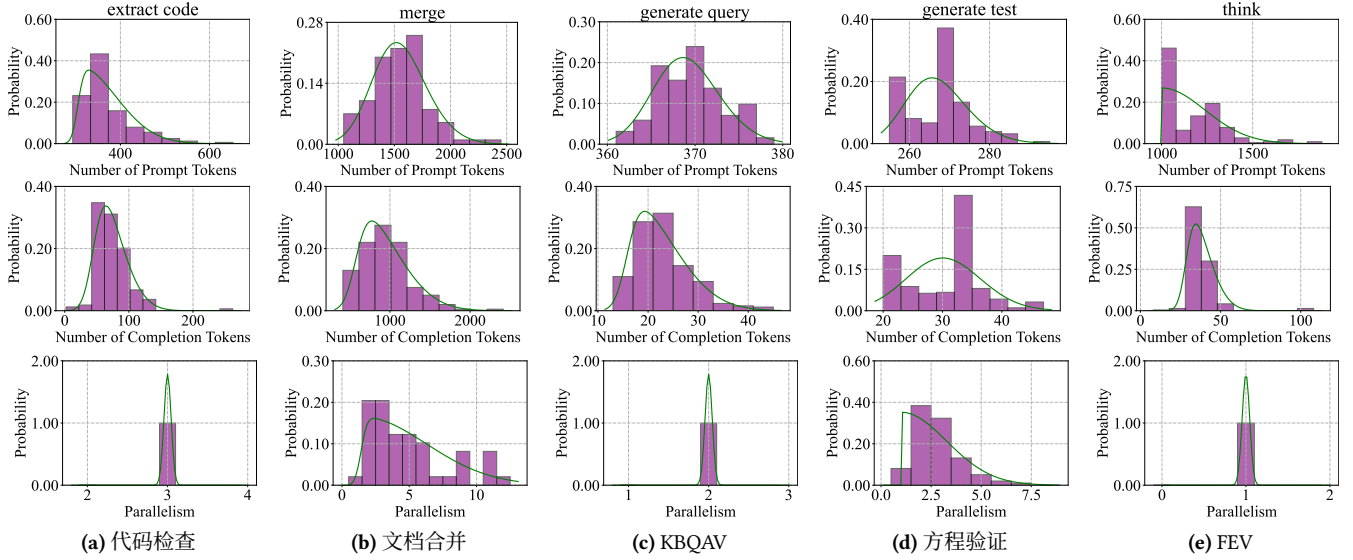


图 3. 五个应用各自进行了超过 100 次试运行；图中展示了从每个应用任意选取的一个请求的提示输入和生成输出 Token 长度分布，该请求已在图顶端标出。对每种情况，我们将长度范围划分为 10 个区间，并计算每个区间中取值出现的概率；同时给出假设为偏态高斯分布时的拟合曲线，供参考。

应用通常会重复执行，因此可以离线进行充分的性能分析。如图 1 所示，每个 LLM 应用由多个功能单元组成：一个 LLM 单元中的任务，例如 DM 应用中的 merge 单元，具有相同的系统提示词；一个非 LLM 单元中的任务需要特定的后端类型，例如 CG 应用中的 execute-code 单元需要特定 Docker 容器。通过程序分析，我们可以记录每个功能单元的执行信息，并将这些信息用于需求估计。

为了具体说明，我们分别执行图 1 中的五个 LLM 应用各 100 次，回放从官方数据集中采样得到的不同输入。图 3 展示了从每个应用任意选取的一个功能单元的执行统计，包括输入/输出 Token 长度以及请求并行度。如图 3 所示，虽然资源需求并非确定值，但在多次试运行之间仍表现出显著稳定性。例如，DM 应用的 merge 请求输出长度约为 1000 个 Token，而 KBQAV 应用的 generate-query 请求输出长度则介于 10 至 50 个 Token 之间。这一现象是合理的，因为需求量在一定程度上是由 LLM 应用使用场景决定的内在属性：merge 请求的输出是一份文档，而 generate-query 请求生成的是简短查询。因此，后者长度分布的均值和离散程度都显著较小。请求并行度方面也存在这种稳定性。

因此，我们可以利用动态程序分析来估计 LLM 应用的资源需求。这些信息能够帮助 LLM 服务系统优化任务排队和后端资源供给。理想情况下，若已知每个 LLM 应用的总执行时间，我们就可以使用最短剩余处理时间 (SRPT) 策略来最小化平均 ACT；若已知接下来使用哪种后端以及使用时刻，就可以计算预热冷后端的最佳时间，在没有资源浪费的情况下，让应用迅速完成。

挑战。 然而，尽管前景良好，要利用 LLM 应用的程序分析取得最佳效率表现，并非易事，主要存在两方面挑战。

一方面，需求动态性和后端多样性是 LLM 应用的固有属性，因此需求建模必须兼顾通用性和准确性。所谓“通用”，是指建模方法需要兼容多种后端以及不同的需求依赖结构；所谓“准确”，是指建模方法需要维持恰当程度的不确定性，尝试缩小估计范围，同时避免过于武断。具体而言，每次应用运行中的功能单元类型和数量，以及任务持续时间，都与运行时输入密切相关，可能显著偏离性能分析结果中的平均情况；仅记录平均分析结果并不准确。

另一方面，基于性能分析的建模只能缓解需求不确定性，无法将其消除，因此难以直接取得最优服务性能。事实上，我们将在后文说明：当调度具有不确定需求的作业时，经典 SRTF 策略已不再最优。因此，我们的队列管理和后端预热方法必须适应这种不确定性，才能取得尽可能好的性能。我们将在后续章节中解决这些挑战。

3 Hermes 设计

3.1 概览

我们提出 Hermes，一个用于高效服务端托管 LLM 应用的系统。Hermes 建立在称为概率需求图 (PDGraph) 的性能建模范式之上；该范式采用离线结合在线的方式，为一般性 LLM 应用提供准确的需求估计 (§3.2)。如图 4 所示，Hermes 维护一个知识库，存储每个应用的性能分析信息。当用户启动一个云端托管的 LLM 应用，并将其任务提交给 HermesScheduler 后，HermesScheduler 会检索该应用的 PDGraph，以辅助调度。具体而言，HermesScheduler 根据每个任务所属应用的优先级调度任务；该优先级依据估计出的需求信息计算 (§3.3)。此外，根据当前执行状态，HermesScheduler 还会在恰当时刻向 HermesLet 发出后端预热信号，以避免冷启动延迟 (§3.4)。

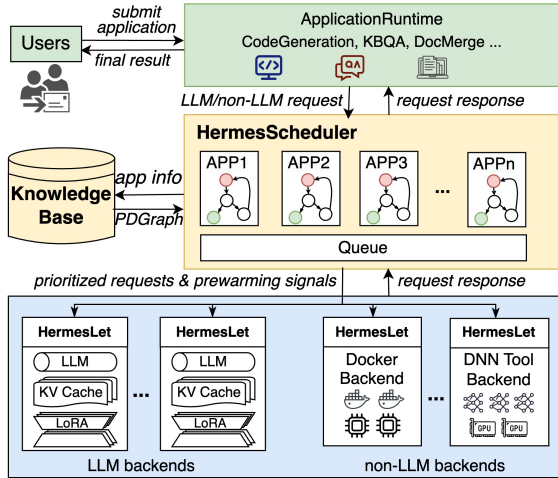


图 4. Hermes 的系统架构。

3.2 使用 PDGraph 进行需求建模

本节旨在为一般性 LLM 应用准确建模需求，以优化队列管理和后端预热策略。回顾图 3，我们发现，同一个功能单元的需求量在不同试运行之间相对稳定。然而，由于交互反应和 LLM 规划等机制，单元之间的结构并非事先确定；同时，单次运行的具体需求量也可能偏离平均情况。因此，在 LLM 应用需求建模中，我们需要覆盖单元间结构的动态性，以保证通用性；同时结合历史执行信息与运行时线索，以保证准确性。为此，我们提出将 LLM 应用建模为由功能单元组成的概率图，并为每个单元节点赋予适当属性。我们将这种建模范式称为概率需求图（PDGraph）。

使用 PDGraph 为一般性 LLM 应用建模需求。 如图 5 所示，在基于 PDGraph 的需求建模范式中，每个应用都被记录为一组功能单元。对于每个功能单元，我们记录三类数据：*backend-spec* 描述资源类型及具体配置；*backend-consumption* 描述该后端上的资源消耗量；*next-unit* 描述具有依赖关系的功能单元之间的概率跳转关系。由此，我们能够为具有不同结构和后端的一般性应用建模资源需求：每种后端，包括 OpenAI function calling [15] 等用户定义后端，都能统一表示为一个 *backend-spec* 条目；静态结构应用也能由 PDGraph 表示，此时任意两个有关单元之间的跳转概率恒为 1。特别地，对于每个 LLM 单元，我们记录输入/输出长度和请求并行度，而非绝对时间，以便适应 A100、H100 等不同运行时执行平台。这些属性能够支持多个方面的效率优化：记录后端消耗量，有助于优先处理较短的应用；记录后端具体信息和跳转关系，有助于提前预热即将调用的后端。

此外，为了处理动态的后端消耗量和概率跳转关系，我们将分析得到的资源需求表示为分布，而非单个数值。具体而言，每次性能分析运行结束后，我们将每个功能单元的執行信息追加到列表中。对于动态后端消耗量，我们发现，记录原始取值优于记录拟合偏态正态分布的系数，其原因包括描述保真度和计算效率：真实分布实

```
LLM-application:
  unit-1:
    backend-spec:
      LLM: LLaMA-3
    backend-consumption:
      input-length: [879,764 ...] # List
      output-length: [210,355 ...] # List
      parallelism: [5,3 ...] # List
      significance-mask: [T, F, F, T, F]
    next-unit:
      unit-1: 0.75 # Probability
      unit-2: 0.25
  unit-2:
    backend-spec:
      docker: python:3.10-slim
    backend-consumption:
      duration: [7.6s,5.4s ...] # List
    next-unit:
      unit-3: 1.0 # Probability
  ...
  unit-N: ...
```

图 5. 一个 LLM 应用的 PDGraph 示例。

际上并不规则，而拟合系数以及支持后续计算都耗时较长，最高可达数秒。对于概率跳转关系，我们将历史跳转频率计算为分支选择概率。同时，我们将记录值的最大数量设置为 1000，并采用 FIFO 方式淘汰；其存储成本实际上可以忽略不计。

进一步地，我们使用蒙特卡洛方法估计整个应用的总需求。具体而言，我们沿 PDGraph 进行随机游走，对下游分支和单元需求值进行采样，直到收集到足够数量的样本。由于典型 PDGraph 的节点数量和列表大小都有限，这一过程可以高效完成。将 LLM 执行时间与非 LLM 执行时间相加，就能够估计一个应用的总执行成本；其中，输入/输出 Token 长度根据运行环境中的平均单 Token 处理时间，转换为绝对服务时间。

通过在线细化估计提高准确性。 准确的需求估计对于 LLM 应用的整体服务效率至关重要。然而，直接使用历史执行信息并不充分：它只是先验知识，而运行时执行信息，即后验知识，对于细化估计结果同样有价值。事实上，由于各阶段之间存在结构依赖，某一阶段的资源需求往往与上游阶段相关。具体而言，我们注意到，通常存在三种跨单元需求相关模式：

- 一个单元的输入长度，可能与上游单元的输入/输出长度相关。例如，对于 DM 应用，scoring 单元中每个请求的输入，都是上游 aggregate 单元中一个请求输出的超集，额外包含固定的系统提示词。同时，对于循环单元，下游请求与上游请求共用同一提示模板，因此输入长度相似。
- 一个单元的输出长度，可能与其自身输入长度以及上游单元的输出长度相关。例如，对于 CG 应用中的 generate-code 单元，更复杂的输入任务通常对应更长的提示词，也往往生成更长的代码片段。同样，循环单元中各次请求的输出长度也较为相似。
- 一个单元的并行度，可能与上游单元的并行度相关。例如，在 KBQAV 应用中，上游 generate-queries 单元的每个推理请求，都会在下流 verify-claim 单元中触发一个对应的推理请求。

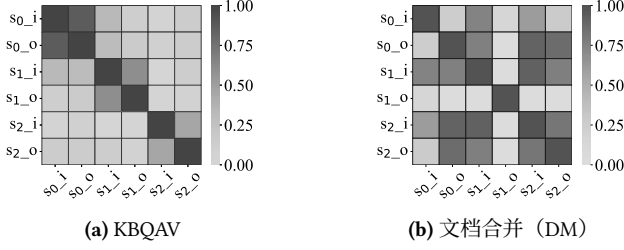


图 6. 具有依赖关系的功能单元之间的需求 Pearson 相关系数。我们分别展示 KBQAV 和文档合并应用中的三个单元，其中 s_{x-i} 、 s_{x-o} 表示第 x 个单元的输入/输出 Token 长度。

为了验证上述相关类型是否存在,我们基于超过 100 次试运行的分析数据,使用 *Pearson* 相关分析[66]。我们将需求范围划分为 10 个区间,并使用 $P(X = i)(i = 0, 1, \dots, 9)$ 表示需求量变量 X 落入第 i 个区间的概率。随后,将两个需求变量 X 和 Y 之间的 Pearson 相关系数计算为 $\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{E((X-\mu_X)(Y-\mu_Y))}{\sigma_X \sigma_Y}$ 。在图 6 中,我们进一步展示 KBQAV 和文档合并应用中任意两个需求变量之间的 Pearson 相关系数;每个变量代表一个单元的输入或输出长度。对于 KBQAV,每个单元的输出长度与自身输入长度高度相关;而对于 DM,每个单元的输入长度则与上游单元的输入长度高度相关。

图 6 还表明,每个应用都具有不同的相关模式;为了提高预测效率,我们只考虑相关系数 ρ 大于 0.5 的需求相关关系。为此,我们在每个单元中增加一个五元组 ($M_I^{\tilde{I}}, M_O^{\tilde{O}}, M_P^{\tilde{P}}, M_I^{\tilde{I}}, M_O^{\tilde{O}}, M_P^{\tilde{P}}$), 用作掩码,表示相应需求相关关系是否成立: M_Y^X 表示需求变量 X 是否影响 Y ; I 、 O 和 P ($\tilde{I}$ 、 $\tilde{O}$ 和 $\tilde{P}$) 分别表示当前单元(上游单元)的输入长度、输出长度和请求并行度。

上述相关分析使在线需求预测更加精确。一个单元完成后,其执行信息可以立即用于预测未来单元的需求。此时,我们面对的是一个条件预测问题。假设刚完成单元的请求输入长度和输出长度分别落在区间 i 和 o , 且二者均与下游单元的输入长度相关,则需要预测 $P(I|\tilde{I} = I, \tilde{O} = o)$ 。为此,我们连接两个具有依赖关系的功能单元的历史执行记录,即从每次试运行中分别得到 $(\tilde{I}, \tilde{O}, I)$ 元组,再筛选出满足 $\tilde{I} = i$ 且 $\tilde{O} = o$ 的分析元组。筛选后记录中的 I 分布用于指导蒙特卡洛采样,从而更准确地估计当前这一轮的资源需求。

3.3 基于 PDGraph 的队列管理

借助 PDGraph, 我们现在能够获得 LLM 应用整体资源需求的概率估计。本节探索如何利用这些需求估计来优化整体排队性能。

基于 Gittins 策略优化平均 ACT。 对于 LLM 应用的高效调度, 一个主要指标是平均应用完成时间 (ACT)。经典的最短剩余处理时间 (SRPT) 算法已被证明在这一目标下最优 [64]。然而, 直接使用 SRPT 并不适合我们的问题: SRPT 依赖确定的需求知识来做出调度决策, 但 LLM 应用的资源需求事先并不确定。我们的 PDGraph 策略缓

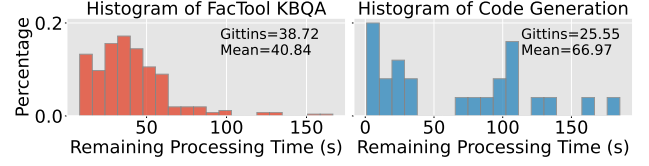


图 7. 给定两个正在运行的 LLM 应用 FacTool_KBQA 和 Code_Generation, SPRT 会先调度前者, 而 Gittins 策略则先调度后者, 因为后者具有更小的 Gittins 排序值。就在 9 秒内完成的概率而言, FacTool_KBQA 为 0.3%, 而 Code_Generation 为 16%; 这说明在实际情况下, 优先处理 Code_Generation 的确能够获得更高回报。

解了这种不确定性, 却没有将其完全消除。虽然可以使用需求分布的期望值, 即均值, 来模拟 SRPT 调度, 但这种方法忽略了当前执行进度, 因此往往表现不佳。例如, 一个期望输出长度为 20 个 Token 的请求, 即使已经生成了 100 个 Token, 也可能仍未停止生成。在这种情况下, 我们需要根据最新进度及时更新需求估计, 而不能始终坚持先验期望; 否则, 期望执行时间减去已执行时间所得到的剩余处理时间, 竟可能成为负数。因此, 我们在设计调度算法时, 需要引入对不确定性的感知。

我们现在面临的问题, 是针对持续时间未知但持续时间分布已知的作业, 最小化平均完成时间。文献已经研究过这一问题, 且 Gittins 策略 [16, 34, 65] 已被证明最优。Gittins 策略的基本思想, 是根据每个作业至今已执行的时间及其大小分布, 计算一个 Gittins 排序值, 作为调度优先级。具体而言, 设一个应用的持续时间分布为 $\mathcal{D}$, 且已经执行了时长 a , 则其 Gittins 排序值 G 可以表示为:

$$G(\mathcal{D}, a) = \inf_{\Delta > 0} \frac{E[\min\{X_{\mathcal{D}} - a, \Delta\} | X_{\mathcal{D}} > a]}{P\{X_{\mathcal{D}} - a \leq \Delta | X_{\mathcal{D}} > a\}}, \quad (1)$$

其中, $X_{\mathcal{D}}$ 是服从 $\mathcal{D}$ 的随机变量。给定服务预算 Δ , 即从现在起允许额外生成的 Token 数量, 式 1 中的分母表示生成过程能在 Δ 之前结束的概率, 有助于缩短平均完成时间; 分子表示对应的资源消耗, 即服务该请求所付出的成本。为了确定调度优先级, Gittins 指数实质上计算在任意可能的成本预算下, 能够达到的最大成本与回报之比, 已有工作 [34, 65] 表明, 该方法在理论上能够取得最小平均完成时间。图 7 展示了一个例子: Gittins 策略取得了不同于 SPRT、且更好的调度顺序。此外, 为了提高计算效率, 实际实现中, 我们仅在每经过一个区间时段之后更新 Gittins 指数估计。

扩展至其他效率¹目标。 对于高效 LLM 应用调度, 除了 ACT 指标之外, 用户也可能为应用请求指定一个截止时间, 并期望在这一时刻之前获得最终输出。此时, 效率指标是能够满足截止时间要求的应用比例。对于基于截止时间的调度, 我们注意到, PDGraph 需求建模同样能够帮助取得更好的性能, 超过最早截止时间优先 (EDF) [20] 等不感知需求的调度方法。EDF 总是优先处

¹本文主要关注 LLM 应用调度的效率方面; 但利用 PDGraph 提供的请求信息, 实际上还可以启用感知需求的公平调度方法, 例如加权公平排队 [21], 从而增强公平性。

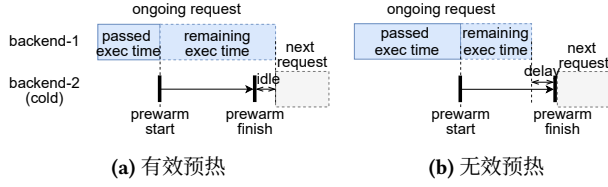


图 8. 设置预热触发时刻时的权衡：过早预热会浪费资源，而过晚预热会延迟应用完成。

理截止时间最早的应用；但截止时间更早并不意味着更紧迫，因为紧迫性还取决于需求量。基于 PDGraph，我们将相对于截止时间的最坏情况松弛时间 S 定义为

$$S(\mathcal{D}, a) = t_{ddl} - t_{now} - (\sup X_{\mathcal{D}} - a), \quad (2)$$

其中， S 表示在最坏情况下，距离截止时间还剩余的可用时间。随后，我们按照 S 从小到大的顺序优先处理应用，模拟最小松弛时间优先 (LSTF) 策略 [29]。

3.4 基于 PDGraph 的后端预热

如 §2.2 所述，冷后端会延迟 LLM 应用的完成。利用 PDGraph，Hermes 可以估计下游请求的到达以及它们需要的后端类型。在当前单元完成之前，提前预热下游单元将要使用的冷后端，就能够将预热延迟从关键路径中消除。然而，后端预热并非没有成本。一方面，一个功能单元可能具有多个下游单元，预热的后端有可能始终不会被使用；另一方面，当前单元可能比预期更晚完成，意味着已经预热的后端会长时间闲置等待。因此，如图 8 所示，决定何时触发预热时存在权衡：更积极的预热有助于让应用更快完成，但代价是更大的资源浪费。

于是，我们引入称为期望预热有效性的可调参数 K ，用于控制这种权衡。具体而言，给定一个正在运行的功能单元，如果它的某个下游后端尚未激活，我们就采用分析方法，确定是否以及何时触发预热。设 p_s 为选择目标单元的概率， t_c 为当前单元的实际完成时刻， t_s 为开始预热的时刻， t_p 为预热操作的持续时间。则预热有效的概率，即目标下游请求最终到达时，其后端已经完成预热的概率，可以表示为

$$p_e = p_s * P(t_c > t_s + t_p). \quad (3)$$

给定参数 K ，若 $p_s < K$ ，Hermes 不触发预热；否则，它在使 $p_e = K$ 成立的时刻触发预热。这种参数 K 可以视为一种服务等级约定：更重要的应用，例如高级用户提交的应用，可以分配较小的 K 值。

上述预热技术适用于 Docker、DNN 等非 LLM 后端，也适用于 LLM 后端中的缓存资源。事实上，LLM 服务器上的缓存空间对于高效 LLM 服务十分重要。已有工作 [17, 47] 表明，对于共享相同提示 Token 的请求，KV cache 可以显著提高 Token 生成速度。同时，低秩适配 (LoRA) [38] 被广泛用于支持采用定制模型的 LLM 服务；这些 LoRA 适配器需要不可忽略的加载时间，因此也应缓存在内存中 [48] (2.2)。因此，提前加载所需的 KV cache 或 LoRA，同样是高效执行 LLM 应用的必要条件；我们还将在此 §5.4 中评估这一点。

4 实现

我们使用 4100 行 Python 代码实现了 Hermes。如图 4 所示，我们的实现包含两个关键组件：HermesScheduler 和 HermesLet。我们使用 vLLM 0.4.3 [47] 作为底层推理引擎，并使用基于 ZeroMQ 的 RPC [14] 在 HermesScheduler 和 HermesLet 之间交换控制消息。此外，我们使用 11,300 行 Python 代码构建了全面的基准套件，覆盖图 1 中的全部应用类型。

在构建 PDGraph 模型时，我们对每个应用进行 1000 次性能分析，并将其 PDGraph 存储在 JSON 文件中。每个应用的 JSON 文件平均大小约为 100KB。加载该 JSON 文件后，HermesScheduler 会启动一个后台 RPC 线程，根据 HermesLet 报告的最新执行状态，以每个区间为粒度，持续更新需求估计。HermesScheduler 据此更新应用调度优先级，依据为 Gittins 指数或最坏情况松弛时间；优先级一旦变化，就通知 HermesLet。HermesScheduler 还在所需时刻，向 HermesLet 发出后端预热信号。HermesLet 同样维护专用的后台 RPC 客户端，以持续与 HermesScheduler 通信；它通过该客户端报告执行状态，同时接收更新后的请求优先级和预热信号。它还使用两个专用后台线程，按照 HermesScheduler 的决策，预取 KV cache 块，或 LoRA 适配器，并预热 Docker 容器。

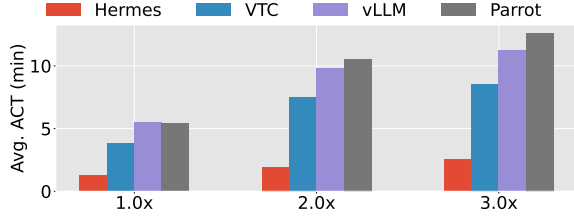
5 评估

5.1 总体设置

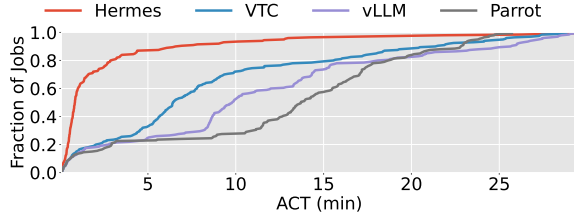
硬件平台。 我们分别通过单 GPU 和多 GPU 实验评估 Hermes。单 GPU 评估使用配备一块 A100 GPU 的服务器；多 GPU 评估使用配备八块 H800 GPU 的服务器。单 GPU 服务器配备四颗 16 核 AMD EPYC 7302 CPU、128GB DRAM、4TB SSD，以及一块 NVIDIA A100-PCIe-40GB GPU。多 GPU 服务器配备四颗 48 核 Intel Xeon Platinum 8558 CPU、2TB DRAM、28TB SSD，以及八块 NVIDIA H800-80GB GPU。除非另有说明，我们将 LLaMA2-7B 和 LLaMA2-13B 部署在单块 A100-PCIe-40GB GPU 上，并使用四路张量并行，将两个 LLaMA3-70B 模型部署在八块 H800-80GB GPU 上，以进一步评估 Hermes 在更大规模下的性能。

工作负载。 为了研究 Hermes 在真实环境中的性能，我们从 MoonCake [60] 发布的轨迹中采样请求到达时间分布；该轨迹能够清楚呈现真实场景中的突发到达模式。我们采用其到达时间分布，同时从图 1 所示应用族中采样真实输入。具体而言，与已有工作 [41, 59, 81] 类似，我们将小型应用 (EV、FEV、CC、ALFWI 和 KBQAV，通常少于 1 分钟)、中型应用 (CG 和 PE，通常介于 1 至 10 分钟) 以及大型应用 (DM 和 MRS，通常超过 10 分钟) 的采样概率分别设置为 72%、26% 和 2%。

基线。 我们将 Hermes 与三种调度策略进行比较：vLLM [13]、Parrot [51] 和 VTC [71]。如 §2.2 所述，vLLM 在请求层面采用 FCFS，Parrot 在应用层面采用 FCFS。VTC 旨在公平服务来自不同用户的任务。对于具有截止时间的场景，我们额外将最早截止时间优先 (EDF) 策略 [20] 作为基线。其他基线将在各个具体微基准实验中介绍。Hermes 的默认超参数设置如下：Pearson 相关系

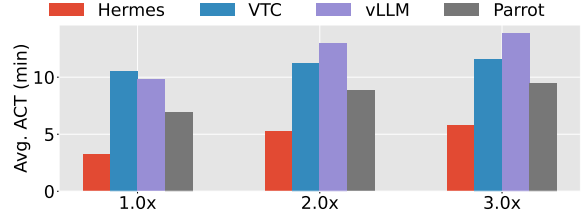


(a) 不同应用到达强度下的平均 ACT。

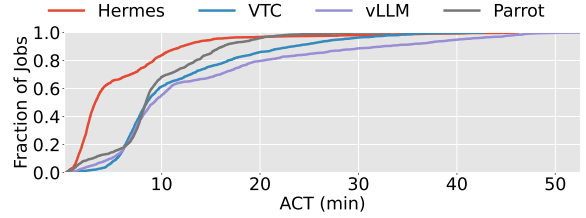


(b) 不同调度器下 ACT 的累积分布函数 (CDF)。

图 9. 单个 LLaMA2-7B 模型运行在一块 A100 GPU 上的实验结果。



(a) 不同应用到达强度下的平均 ACT。



(b) 不同调度器下 ACT 的累积分布函数 (CDF)。

图 10. 两个 LLaMA3-70B 模型运行在八块 H800 GPU 上的实验结果。

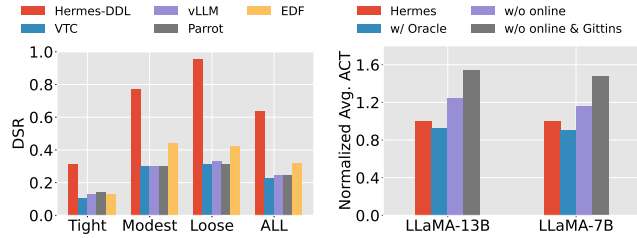


图 11. 不同调度器的截止时间满足率。

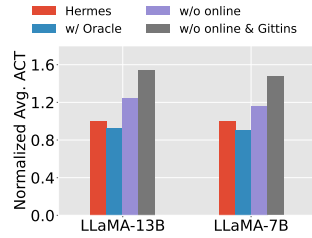


图 12. 针对处理需求动态性的方法进行的消融研究。

数阈值 (§3.2) 设为 0.5, 期望预热有效性参数 K (§3.4) 设为 0.5, 用于描述分布的区间数 (§3.2 和 §3.4) 设为 10。

5.2 端到端调度性能

最小化平均 ACT。 我们首先使用应用平均完成时间, 评估 Hermes 如何提高应用服务效率。我们进行了两组实验, 分别运行 LLaMA2-7B 和 LLaMA3-70B; 根据不同硬件的计算能力, 相应提交 300 个和 3000 个应用。我们在 10、15 和 30 分钟的提交窗口下评估性能, 对应工作负载强度为 3 $\times$ 、2 $\times$ 和 1 $\times$, 并报告 ACT 分布。如图 9a 和图 10a 所示, 在每种情况下, Hermes 的表现都显著优于 vLLM、Parrot 和 VTC。例如, 在 LLaMA2-7B 实验中, 工作负载强度为 1 $\times$ 时, Hermes 的 ACT 相对于 vLLM 降低 77.0%, 相对于 Parrot 和 VTC 分别降低 76.7% 和 66.7%; 在 LLaMA3-70B 实验中, 工作负载强度为 3 $\times$ 时, 对应降低幅度分别为 58.5%、39.1% 和 50.3%。对于应用尾部性能, 在 LLaMA2-7B 实验中, 工作负载强度为 1 $\times$ 时, Hermes 的 P95 ACT 相对于 vLLM 降低 82.4%, 相对于 Parrot 和 VTC 分别降低 69.0% 和 74.1%; 在 LLaMA3-70B 实验中, 工作

负载强度为 3 $\times$ 时, 对应降低幅度分别为 62.2%、21.7% 和 45.8%。

最大化截止时间满足率。 接下来, 我们评估 Hermes-DDL, 这是专为具有截止时间约束的场景设计的 Hermes 变体, 采用 §3.3 中的方法。沿用 LLaMA2-7B 的实验设置, 我们在 15 分钟提交窗口内提交相同的 300 个应用。此外, 我们通过随机选取原始执行时间的缩放因子, 为每个任务分配不同的截止时间: 1.2 $\times$ 表示紧张, 1.5 $\times$ 表示适中, 2 $\times$ 表示宽松; 这一方法与已有研究 [35, 55] 一致。图 11 报告了截止时间满足率 (DSR), 即能够在指定截止时间之前完成的应用比例; 既包含全部应用的结果, 也包含各个截止时间缩放类别的结果。结果表明, 在所有评估方案中, Hermes-DDL 取得最高 DSR, 相对于 EDF 提高了 1 $\times$ 。这一提升主要来自 Hermes-DDL 利用应用需求信息的能力: 它优先处理最紧迫的应用, 并推迟较不紧迫的应用。

5.3 应对需求动态性的效果

设置。 回顾前文, 为了应对需求动态性, 我们采用了离线性能分析、在线细化以及 Gittins 策略等多种技术。现在, 我们比较四种方案的性能, 以验证各项技术各自的有效性: Hermes、去掉在线细化的 Hermes、同时去掉在线细化和 Gittins 的 Hermes, 以及带有 Oracle 的 Hermes。最后一种方案表示确切需求已知的理想情况, 通过将 temperature 设置为 0 进行试运行来实现。我们分别使用 LLaMA-7B 和 LLaMA-13B 作为 LLM 后端进行该实验, 并在 10 分钟时间窗口内提交 300 个应用。

结果。 图 12 展示了各情况下的平均 ACT, 并以 Hermes 的结果归一化。去掉在线相关分析后, 使用 LLaMA-7B 模型时, 平均 ACT 将增大 15.3%。若进一步禁用 Gittins 策略, 性能下降幅度将增大至 47.5%。这些结果证实, 在

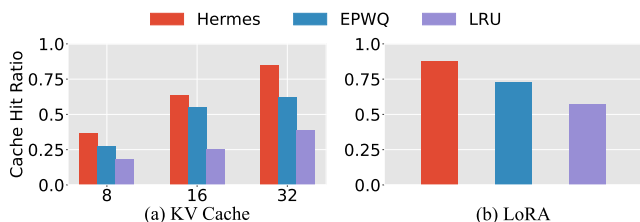


图 13. 不同缓存管理策略下, (a) KV cache 和 (b) LoRA 适配器的缓存命中率。

线细化和 Gittins 策略对于应对需求不确定性都不可或缺。同时, Hermes 与 Hermes-Oracle 之间的性能差距小于 10%, 表明 Hermes 能够为 LLM 应用提供接近最优的调度性能。我们还注意到, 在每种情况下, LLaMA-7B 与 LLaMA-13B 的性能都十分相似。

5.4 后端预热的效果

LLM 后端预热的效果。 如 §3.4 所述, 预热 KV 和 LoRA 缓存能够促进 LLM 服务。为了验证这一点, 我们在 15 分钟内向 A100 服务器提交 500 个应用。我们采用两种缓存管理基线: LRU, 以及基于等待队列的驱逐/预取 (EPWQ) [33, 70]。LRU 以被动换入换出方式进行预热; EPWQ 仅在请求已经进入等待队列时才预热 KV cache, 而此时往往已经太晚。对于每种方法, 我们在不同缓存大小, 即 8GB、16GB 和 32GB 下, 测量整体缓存命中率, 也就是请求到达时 KV cache 已经完成预热的比例。如图 13 (a) 所示, Hermes 始终取得最高缓存效率, 相对于 LRU 和 EPWQ, 整体缓存命中率最高分别提高 1.11× 和 0.33×。相对于 LRU 和 EPWQ, 它使平均 ACT 分别降低 18% 和 6%。

随后, 我们评估预热 LoRA 缓存的收益。我们将 max-loras, 即每次迭代中的最大并行处理数量, 设置为 10; 将 max-cpu-loras, 即 CPU 内存中能够存储的 LoRA 缓存数量, 设置为 20; 并为 LLaMA-7B 使用总计 200 个秩为 8 的 LoRA 适配器。我们提交一个持续 25 分钟、包含 1,000 个应用的工作负载, 每个应用随机分配一个 LoRA 适配器; 同一应用的全部请求使用相同 LoRA。图 13 (b) 分别展示了 LRU、EPWQ 和 Hermes 的缓存命中率。相对于排名第二的 EPWQ, Hermes 能够将缓存命中率提高 21.1%。

非 LLM 后端预热的效果。 为了评估非 LLM 后端中的预热策略, 我们选择 CG 和 PE 两个应用, 它们分别需要 Docker 和 DNN 后端。我们分别在不同预热积极程度, 即不同期望预热有效性超参数 K 下运行每个应用。图 14 展示了预热带来的平均延迟降低以及资源浪费, 验证了非 LLM 后端预热的有效性; 对于 DNN 后端, 我们只分析调用 ViT 或 Diffusion 模型的 PE 应用。此外, 图 14 还表明, 较小的 K 通常能够带来更快完成, 但代价是更多资源浪费, 与 §3.4 中的分析一致。

5.5 开销分析

Hermes 的主要开销在于使用 Gittins 确定应用调度优先级; 每经过一个区间大小对应的时长, 便迭代执行一次。这里的区间用于描述需求分布, 如图 3 所示。图 15a 测量了不同到达率下 Gittins 策略的平均运行时间; 不同到达率对应不同调度规模。结果表明, 每种情况下的调度开销都很小, 少于 3 毫秒。这是因为 PDGraph 模型本身规模不大, 能够高效处理。进一步地, 在图 15b 中, 我们测量了 Gittins 策略运行时间如何随区间数量变化。结果表明, 使用更多区间描述需求分布, 几乎会线性增加策略运行时间, 却无法改善调度性能。

6 相关工作与讨论

加速单个 LLM 请求。 许多工作尝试从多个方面加速单个 LLM 请求。在算子实现方面, FlashAttention [28] 和 FlashDecoding [36] 一类算法, 通过引入对 IO 的感知, 被广泛用于实现高效 LLM 推理。在模型部署方面, 已有模型量化 [31, 46] 和分页注意力 [47] 方法, 用于最大化后端利用率。在推理算法方面, 连续批处理 [79] 和推测解码 [24, 54] 等方法, 也通过加速解码过程来提高推理吞吐量。这些工作与本文方法互补, 可以结合使用。

调度多个 LLM 请求。 近期有些工作注意到, 在请求层面优化调度性能的必要性。FastServe [77] 使用多级反馈队列进行 LLM 推理服务, 但代价是相对较高的抢占开销; Llumnix [73] 和 LoongServe [76] 支持运行时请求迁移, 以改善负载均衡和隔离性。同时, 另一些工作尝试采用类似 SJF 的调度算法 [37, 57, 67]。虽然这些方法能够缩短请求平均完成时间, 但它们不感知应用, 因此如 §2.2 所述, 无法让应用快速完成。此外, Hermes 能够容易地扩展, 以支持多后端路由, 在同一个引擎上共同部署请求时避免相互干扰。例如, 利用 PDGraph 中记录的输入/输出长度信息, 可以将需求互补的请求安排到特定引擎上, 例如具有较长输出的内存密集请求, 以及具有较短输出的计算密集请求。

作业需求预测。 需求预测对于高效作业调度十分重要。在传统领域, 一系列研究已经提出, 通过测试平台性能分析 [19, 80] 或数学建模 [39, 58] 来准确预测性能。然而, LLM 工作负载由于生成式运行方式, 表现出特有的需求不确定性, 而此前的工作尚未捕获这种不确定性。近期, 一些方法提出使用 LLM 本身来预测 LLM 请求的输出 Token 长度 [45, 61, 67, 82], 但代价是耗时的模型微调过程。此外, 这些方法都没有从应用角度预测需求: 由于忽略任务依赖关系, 其预测准确性较低, 同时也无法支持针对多种后端的推测预热。

Hermes 在先进推理架构中的可扩展性。 Prefill 与 decode 分离 [18, 83] 近期成为流行的 LLM 部署范式, 能够避免请求之间的相互干扰。Hermes 能够顺利兼容这种解耦推理架构。一方面, Hermes 的排队算法在全局层面工作, 得到的优先级适用于所有后端; 另一方面, 在 PD 解耦架构中, 每个 prefill 或 decode 实例通常仍需要模型或 KV cache 预热。同时, Hermes 也可以容易地扩展, 以支持多后端路由 [37, 56], 在同一个引擎上共同部署请求

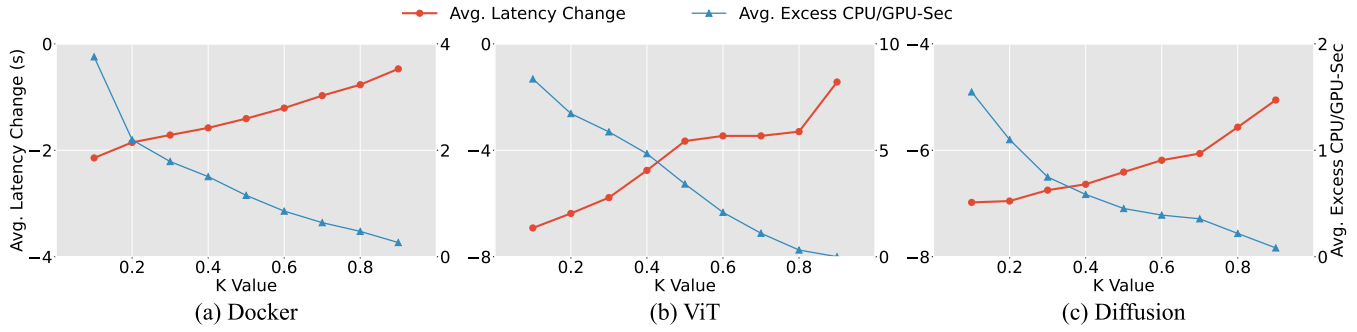


图 14. 非 LLM 后端在不同预热积极程度 K 下的收益与成本。较低的 K 值表示更积极的预热。

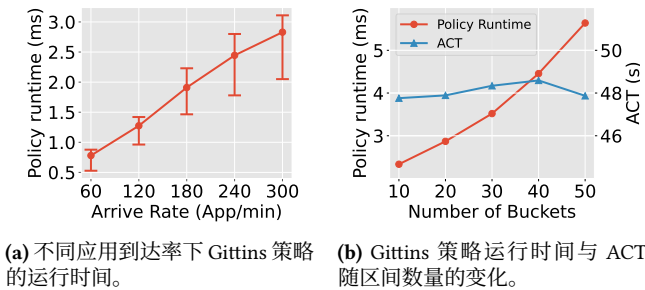


图 15. 在按比例缩放的 MoonCake 轨迹下，Hermes 在不同应用到达率时的调度策略运行时间。

时避免干扰。例如，根据 PDGraph 记录的输入/输出长度信息，可以将需求互补的请求放到特定引擎上，例如长输出的内存密集请求与短输出的计算密集请求。我们将在未来使用 Hermes 探索这些功能。

7 结论

本文提出 Hermes，一个面向 LLM 应用设计的高效服务系统。Hermes 使用概率需求图 (PDGraph) 建模 LLM 应用的资源需求。利用 PDGraph，Hermes 在队列管理中采用 Gittins 策略，以最小化应用平均完成时间；在具有显式截止时间的场景下，则采用 LSTF 算法，以最大化截止时间满足率。Hermes 还利用 PDGraph 模型，确定预热何种后端以及何时预热。针对常见工作负载的实验结果表明，Hermes 能够显著提高 LLM 应用的服务效率，提升幅度超过 70%。

参考文献

- [1] Multi-Agent-as-a-Service. <https://medium.com/data-science/multi-agent-as-a-service-a-senior-engineers-overview-fc759f5bbcf4>, 2024.
- [2] OpenAI Assistants API. <https://platform.openai.com/docs/assistants/overview>, 2024.
- [3] The origin code of alfworld interaction. <https://github.com/ysymyth/ReAct/blob/6bdb3a1fd38b8188fc7ba4102969fe483df8dc9/alfworld.ipynb>, 2024. [Online; accessed 19-Oct-2024].
- [4] The origin code of code checking. <https://github.com/GAIR-NLP/factool/tree/3f3914bc090b644be044b7e0005113c135d8b20f/>

- [5] The origin code of code generation. <https://github.com/microsoft/autogen/tree/0560bdd645dfbc579a71f2f0fea98ea83dd3bb3f?tab=readme-ov-file#quickstart>, 2024. [Online; accessed 19-Oct-2024].
- [6] The origin code of document merging. https://github.com/spcl/graph-of-thoughts/tree/a939a4577c07c80b8ecb194793b5a4169d99b31b/examples/doc_merge, 2024. [Online; accessed 19-Oct-2024].
- [7] The origin code of equation verification. <https://github.com/GAIR-NLP/factool/tree/3f3914bc090b644be044b7e0005113c135d8b20f/factool/math>, 2024. [Online; accessed 19-Oct-2024].
- [8] The origin code of fact extraction and verification. <https://github.com/ysymyth/ReAct/blob/6bdb3a1fd38b8188fc7ba4102969fe483df8dc9/FEVER.ipynb>, 2024. [Online; accessed 19-Oct-2024].
- [9] The origin code of knowledge-based-query-answering verification. https://github.com/GAIR-NLP/factool/tree/3f3914bc090b644be044b7e0005113c135d8b20f/factool/knowledge_qa, 2024. [Online; accessed 19-Oct-2024].
- [10] The origin code of plan-and-execution. <https://github.com/microsoft/JARVIS/tree/c62e0faac76c4a2907cabe2cfe4bbe5f2e613400>, 2024. [Online; accessed 19-Oct-2024].
- [11] The Shift from Models to Compound AI Systems. <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>, 2024. [Online; accessed 21-September-2024].
- [12] The tutorials of how to implement mapreduce summarization. <https://python.langchain.com/v0.2/docs/tutorials/summarization/#go-deeper-1>, 2024. [Online; accessed 19-Oct-2024].
- [13] vLLM: Easy, fast, and cheap LLM serving for everyone. <https://docs.vllm.ai/en/stable/>, 2024. [Online; accessed 19-July-2024].
- [14] Zeromq - an open-source universal messaging library. <https://platform.openai.com/docs/overview>, 2024. [Online; accessed 19-Oct-2024].
- [15] OpenAI Function Calling. <https://platform.openai.com/docs/guides/function-calling>, 2025. [Online; accessed 10-Jan-2025].
- [16] Samuli Aalto, Urtzi Ayesta, and Rhonda Righter. On the gittins index in the m/g/1 queue. *Queueing Systems*, 63:437–458, 2009.
- [17] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. Intercept: Efficient intercept support for augmented large language model inference. In *Forty-first International Conference on Machine Learning*, 2024.
- [18] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *USENIX OSDI*, 2024.
- [19] Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. {CherryPick}: Adaptively unearthing the best cloud configurations for big data analytics.

- In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 469–482, 2017.
- [20] Matthew Andrews. Probabilistic end-to-end delay bounds for earliest deadline first scheduling. In *IEEE INFOCOM*, 2000.
 - [21] Jon CR Bennett and Hui Zhang. Wf/sup 2/q: worst-case fair weighted fair queueing. In *Proceedings of IEEE INFOCOM'96. Conference on Computer Communications*, volume 1, pages 120–128. IEEE, 1996.
 - [22] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17682–17690, 2024.
 - [23] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
 - [24] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.
 - [25] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
 - [26] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.
 - [27] I Chern, Steffi Chern, Shiqi Chen, Weizhe Yuan, Kehua Feng, Chunting Zhou, Junxian He, Graham Neubig, Pengfei Liu, et al. Factool: Factuality detection in generative ai—a tool augmented framework for multi-task and multi-domain scenarios. *arXiv preprint arXiv:2307.13528*, 2023.
 - [28] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.
 - [29] Robert I Davis, Ken W Tindell, and Alan Burns. Scheduling slack time in fixed priority pre-emptive systems. In *1993 Proceedings Real-Time Systems Symposium*, pages 222–231. IEEE, 1993.
 - [30] Luciano Floridi and Massimo Chirriatti. Gpt-3: Its nature, scope, limits, and consequences. *Minds and Machines*, 30:681–694, 2020.
 - [31] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
 - [32] Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Aurick Qiao, and Hao Zhang. Efficiently serving llm reasoning programs with certain index. *arXiv preprint arXiv:2412.20993*, 2024.
 - [33] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.
 - [34] John Gittins, Kevin Glazebrook, and Richard Weber. *Multi-armed bandit allocation indices*. John Wiley & Sons, 2011.
 - [35] Diandian Gu, Yihao Zhao, Yinmin Zhong, Yifan Xiong, Zhenhua Han, Peng Cheng, Fan Yang, Gang Huang, Xin Jin, and Xuanzhe Liu. Elasticflow: An elastic serverless training platform for distributed deep learning. In *ACM ASPLOS*, 2023.
 - [36] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Yuhang Dong, Yu Wang, et al. Flashdecoding++: Faster large language model inference with asynchronization, flat gemm optimization, and heuristics. *Proceedings of Machine Learning and Systems*, 6:148–161, 2024.
 - [37] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.
 - [38] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
 - [39] Qinghao Hu, Meng Zhang, Peng Sun, Yonggang Wen, and Tianwei Zhang. Lucid: A non-intrusive, scalable and interpretable scheduler for deep learning training jobs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 457–472, 2023.
 - [40] Redwan Ibne Seraj Khan, Kunal Jain, Haiying Shen, Ankur Mallick, Anjaly Parayil, Anoop Kulkarni, Steve Kofsky, Pankhuri Choudhary, Renée St Amant, Rujia Wang, et al. Ensuring fair llm serving amid diverse applications. *arXiv e-prints*, pages arXiv–2411, 2024.
 - [41] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, and Gregory R Ganger. Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In *ACM SOSP*, 2023.
 - [42] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
 - [43] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. Towards mitigating llm hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 1827–1843, 2023.
 - [44] Hanlei Jin, Yang Zhang, Dan Meng, Jun Wang, and Jinghua Tan. A comprehensive survey on process-oriented automatic text summarization with exploration of llm-based methods. *arXiv preprint arXiv:2403.02901*, 2024.
 - [45] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. s³: Increasing gpu utilization during generative inference for higher throughput. *Advances in Neural Information Processing Systems*, 36:18015–18027, 2023.
 - [46] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*, 2023.
 - [47] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
 - [48] Suyi Li, Hanfeng Lu, Tianyuan Wu, Minchen Yu, Qizhen Weng, Xusheng Chen, Yizhou Shan, Binhang Yuan, and Wei Wang. Caraserve: Cpu-assisted and rank-aware lora serving for generative llm inference. *arXiv preprint arXiv:2401.11240*, 2024.
 - [49] Yinheng Li, Shaofei Wang, Han Ding, and Hang Chen. Large language models in finance: A survey. In *Proceedings of the fourth ACM international conference on AI in finance*, pages 374–382, 2023.
 - [50] Bin Lin, Tao Peng, Chen Zhang, Minmin Sun, Lanbo Li, Hanyu Zhao, Wencong Xiao, Qi Xu, Xiafei Qiu, Shen Li, et al. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache. *arXiv preprint arXiv:2401.02669*, 2024.
 - [51] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of llm-based applications with semantic variable. In *USENIX OSDI*, 2024.
 - [52] Aixun Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
 - [53] Christos Makridakis. The impact of generative artificial intelligence on artists. Available at SSRN, 2025.

- [54] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781*, 1(2):4, 2023.
- [55] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *USENIX OSDI*, 2020.
- [56] Chengyi Nie, Rodrigo Fonseca, and Zhenhua Liu. Aladdin: Joint placement and scaling for slo-aware llm serving. *arXiv preprint arXiv:2405.06856*, 2024.
- [57] Archit Patke, Dharmath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. Queue management for slo-oriented large language model serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*, SoCC '24, page 18–35, New York, NY, USA, 2024. Association for Computing Machinery.
- [58] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, and Chuanxiong Guo. Optimus: an efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference*, pages 1–14, 2018.
- [59] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021.
- [60] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *arXiv preprint arXiv:2407.00079*, 2024.
- [61] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew T Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. Efficient interactive llm serving with proxy model-based sequence length prediction. *arXiv preprint arXiv:2404.08509*, 2024.
- [62] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [63] Shuo Ren, Pu Jian, Zhenjiang Ren, Chunlin Leng, Can Xie, and Jiajun Zhang. Towards scientific intelligence: A survey of llm-based scientific agents. *arXiv preprint arXiv:2503.24047*, 2025.
- [64] Linus Schrage. A proof of the optimality of the shortest remaining processing time discipline. *Operations Research*, 16(3):687–690, 1968.
- [65] Ziv Scully and Mor Harchol-Balter. The gittins policy in the m/g/1 queue. In *2021 19th International Symposium on Modeling and Optimization in Mobile, Ad hoc, and Wireless Networks (WiOpt)*, pages 1–8. IEEE, 2021.
- [66] Philip Sedgwick. Pearson’s correlation coefficient. *Bmj*, 345, 2012.
- [67] Rana Shahout, Eran Malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. Don’t stop me now: Embedding based scheduling for llms. *arXiv preprint arXiv:2410.01035*, 2024.
- [68] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36, 2024.
- [69] Zhuocheng Shen. Llm with tools: A survey. *arXiv preprint arXiv:2409.18807*, 2024.
- [70] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. S-lora: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285*, 2023.
- [71] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 965–988, 2024.
- [72] Yuki Sonoda, Ryo Kurokawa, Yuta Nakamura, Jun Kanzawa, Mariko Kurokawa, Yuji Ohizumi, Wataru Gono, and Osamu Abe. Diagnostic performances of gpt-4o, claude 3 opus, and gemini 1.5 pro in “diagnosis please” cases. *Japanese Journal of Radiology*, pages 1–5, 2024.
- [73] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *USENIX OSDI*, 2024.
- [74] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [75] Olga Vrousseau. Code execution is now by default inside docker container. <https://microsoft.github.io/autogen/0.2/blog/2024/01/23/Code-execution-in-docker/>. [Online; accessed 19-March-2025].
- [76] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 640–654, 2024.
- [77] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.
- [78] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [79] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. ORCA: A Distributed Serving System for Transformer-Based Generative Models. In *USENIX OSDI*, 2022.
- [80] Peng Zheng, Wendi Feng, Arvind Narayanan, and Zhi-Li Zhang. Nfv performance profiling on multi-core servers. In *2020 IFIP Networking Conference (Networking)*, pages 91–99. IEEE, 2020.
- [81] Pengfei Zheng, Rui Pan, Tarannum Khan, Shivaram Venkataraman, and Aditya Akella. Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning. In *USENIX NSDI*, 2023.
- [82] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. Response length perception and sequence scheduling: An llm-empowered llm inference pipeline. *Advances in Neural Information Processing Systems*, 36, 2024.
- [83] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *USENIX OSDI*, 2024.